Resuelve los siguientes ejercicios y entrega el código fuente java tanto en un fichero .pdf como en ficheros de texto, en el .pdf capturas de pantalla (si se piden) y las respuestas a las preguntas que se plantean, si las hay.

EJERCICIOS PADAWAN (OBLIGATORIOS)

EJERCICIO 1. HACER UNA CLASE PARA UTILIZARLA EN OTRAS.

Una carta o naipe Español tiene 2 datos: palo y valor. Crea una clase immutable para representar un naipe español teniendo en cuenta que el valor puede ser: 1, 2, 3, 4, 5, 6, 7, 10, 11 y 12 y el palo puede ser bastos, copas, espadas y oros. Las operaciones que debe tener una carta son:

- Construirse pasando el valor y el palo (si el valor o el palo son icorrectos, lanzar una excepción java.lang.IllegalArgaumentException).
- Getters() (pero no tendrá setters, porque queremos que sea inmutable).
- Sobreescribe toString(): debes tener en cuenta que: el valor 1 se visualiza como "AS", el 10 como "SOTA", el 11 como "CABALLO" y el 12 como "REY". Por tanto la carta 1 de copas debe escribirse como el string "[AS de copas]" el 2 como "[2 de copas]", etc.

EJERCICIO 2. RELACIONES ENTRE CLASES (HERENCIA Y COMPOSICIÓN).

Crea una nueva clase llamada **CartaBrisca** a partir de la clase **Carta** que tenga además un dato **puntos** que fijas en el constructor CartaBrisca(vaor,palo) que llama al constructor de su clase padre y luego de valor al entero puntos (el as tendrá 11, el 3 tendrá 10, la sota 2 puntos, el caballo 3 y el rey 4 puntos). El resto de cartas tendrá 0 puntos. Pon un getter a este dato.

Haz otra clase llamada **BarajaESP** para crear una baraja de cartas españolas (o mazo de cartas). Una baraja española tiene 40 cartas/naipes. Las operaciones que podrá realizar una baraja:

- **Construirse**: un solo constructor sin parámetros, crea e inicializa un array de 40 cartas (AS, 2, 3, 4, 5, 6, 7, sota, caballo y rey) de cada palo (bastos, copas, espadas y oros). Inicializa un entero que indique las cartas que aún quedan sin usar. Imagina que la carta en la posición 0 del array representa la carta en el fondo del mazo de cartas. Cuando se extrae una carta del mazo, se comienza a coger por arriba, así que el entero apuntará a la posición de donde sacar la siguiente carta.
- **baraja()**: cambia de posición las cartas del mazo de forma aleatoria, para que no se sepa la posición de cada una.
- **hayCartas()**: devuelve true si tiene cartas sin usar, false si el mazo está vacío (es decir, se han usado todas).
- **sacaCarta()**: devuelve la primera carta que está en la baraja (la de posición más alta sin usar) y la quita del mazo. Si no quedan cartas sin usar, lanza una excepción.
- **dimeCarta(i)**: devuelve una copia de la carta sin usar que hay en la posición i-ésima o null si no existe carta sin usar en esa posición. Pero no la saca del mazo. Si no quedan cartas in usar, devuelve null.
- **toString()**: convierte a String la baraja de cartas "[[carta1] [carta2] ....]".

EJERCICIO 3 (1 PUNTO). UN JUEGO DE CARTAS.

¿Sabes jugar a la brisca? Es un juego de cartas. Te explico las reglas para el caso de 2 jugadores, porque debes implementar clases llamada Brisca y JugadorBrisca que encapsule el funcionamiento de este juego:

Juegan 2 jugadores (programa y humano) y gana el que más puntos saca. La clase debe almacenar. Cada jugador tiene los puntos que lleva ganados y un máximo de 3 cartas con las que jugar cada mano. Si el jugador es el programa, escoge las cartas al azar. Si es el humano, se le deben enseñar sus cartas, la muestra y si sale en segundo lugar, la carta que ha jugado el otro jugador. Luego se le da la opción de elegir la carta que quiere jugar.

La clase Brisca debe tener:

- Dos jugadores: uno humano y otro el programa.
- Turno: a cuál de los jugadores le toca jugar.
- Una baraja española.
- Muestra: La última carta de la baraja (la de la posición 0). No hay que sacarla del mazo, simplemente hacerla visible a los jugadores.

MECÁNICA DEL JUEGO:

PASO 1. Se baraja el mazo de cartas para que queden al azar.
PASO 2. Se reparten 3 cartas del mazo a cada jugador para formar una mano.
PASO 3. La carta del fondo del mazo se elige como muestra (no se saca del mazo, pero se hace visible a los jugadores).
PASO 4. Se decide el turno al azar: saber que jugador comienza a jugar.
PASO 5. **MIENTRAS** los jugadores tengan cartas **HAZ**
PASO 5.1 El jugador que tiene el turno tira una de sus cartas (el programa elige una al azar y el humano la escoge de entre las que tenga en su mano).
PASO 5.2 El que gana la mano se suma los puntos de las cartas ganadas (ver Nota 1).
PASO 5.3. El que gana se queda con el turno y le toca salir de nuevo.
PASO 5.4. Si hay cartas en el mazo, el que gana coge una carta del mazo (roba), luego coge otra el segundo jugador.
FIN MIENTRAS.
PASO 6. Gana el jugador que más puntos tenga: la partida ha acabado cuando ya no queden cartas por jugar.

Nota 1: ¿Qué carta gana sobre otra? En el caso de la brisca, el orden en que se ponen en juego es importante. Gana:

- La primera carta que se lanza si:
  - La segunda es del mismo palo pero inferior valor (el orden de mayor a menor valor en la brisca es: AS > 3 > REY > CABALLO > SOTA > 7 > 6 > 5 > 4 > 2).
  - La primera es del mismo palo que la muestra y la segunda no.
  - Si son de distinto palo y la segunda no es muestra.
- La segunda carta que se lanza en cualquier otro caso.

Crea un método main() en la clase que permita jugar una partida de Brisca con el ordenador usando la consola de texto para comunicarte con el jugador humano. Completa el constructor,

getters(), setters() u otros métodos que necesites para realizar el programa.

EJERCICIO 4. HERENCIA Y POLIMORFISMO.

Nos piden hacer un programa que gestione las nóminas de los empleados de una empresa. Los empleados deben tener de forma obligatoria: nombre, nace y contrato (fecha de nacimiento y fecha de contratación, puedes usar java.time.LocalDateTime), salarioBase y nominaActual. Un empleado no puede ser menor de 18 años en el momento de ser contratado. El método getSalario() que calcula y actualiza el campo nominaActual y devuelve el valor será abstracto porque según el tipo de empleado se calculará de una forma o de otra.

Tenemos dos tipos de empleados: repartidor y comercial.

- Un comercial, aparte de los atributos anteriores, tiene otro llamado comision (tipo double) que almacena el importe de las comisiones por los productos que ha vendido. Este importe depende de cada producto.
- Un repartidor, aparte de los atributos de empleado, tiene otro llamado numRepartos que contabiliza los repartos que ha realizado desde la última actualización.

También tenemos la clase **Producto**, que tiene un nombre, precio unidad y porcentaje de comisión (0 <= comision < 1) asociado y que cobra cada comercial por producto vendido y que se puede calcular así: importe de comisión al vender 1 unidad del producto = precio * comisión.

También tenemos la clase **Venta** que contiene el comercial que la realiza, el repartidor que la entrega, el producto que se vende y la cantidad de productos vendidos (todos obligatorios). Un booleano que indica si ha sido entregada, inicialmente está a false y se pone a true con un setter. Una venta tiene por tanto dos operaciones públicas:

- El contructor Venta(Producto, cantidad, comercial, repartidor).
- Y setEntregada() que pone entregada a true, calcula y actualiza la comisión del comercial y suma 1 a numRepartos del repartidor.

Crea constructores, getters, setters y sobreescribe toString() (se valora que aproveches la herencia para minimizar la cantidad de código de estos elementos).

El método **getSalario()**, según el tipo de trabajador tendrá una implementación distinta. El método calcula el salario del empleado, actualiza el atributo nominaActual y devuelve el valor de la siguiente manera:

- En un empleado de tipo comercial, modifica nominaActual sumando su salario base y el valor de su comisión y se deja a 0 la comisión.
- En repartidor, se suma en nominaActual su salario base y la cantidad de numRepartos euros dejando a 0 el numRepartos.

Crea una clase ejecutable llamada **Empresa** donde definas un array de empleados de 4 elementos y guardas dos repartidores y dos comerciales.

Repartidor en posición 0:	Pepe, 01/01/1981, 01/01/1999, salario base 800,00
Repartidor en posición 1:	Juan, 02/02/1982, 02/02/2000, salario base 800,00
Comercial en posición 2:	Manuel, 03/03/1983, 03/03/2001, salario base 800,00
Comercial en posición 3:	Laura, 04/04/1984, 04/04/2002, salario base 850,00

Crea dos productos:

Producto 1: Yogur Yoplait, 1,00 € y 0,1 de comisión
Producto 2: PCX 125, 2849,00 € y 0,2 de comisión

Crea una venta donde Pepe entrega 2 unidades del producto 2 que realiza el comercial Manuel. Pon la venta como entregada. Luego recorre el array de empleados y muestra su información con toString() y su nómina con getSalario(). Ejemplo de ejecución:

EJERCICIO 5. JUEGOS DE INFORMACIÓN TOTAL POR TURNOS.

Intentemos hacer programas que sean capaces de jugar una partida de algún juego de forma competente, no con jugadas aleatorias que muchas veces no tendrán sentido para el otro jugador, que acaba por aburrirse al no tener un contrincante competente.

ALGORITMOS MIN-MAX.

Vamos a considerar como programar juegos de 2 jugadores, a los que llamaremos **MAX** y **MIN** por motivos que pronto se harán evidentes. **MAX** normalmente es el programa, **MIN** es un humano. Juegan por turnos hasta que el juego se termina. Puede terminar porque alguno de ellos gane, o no se pueda continuar jugando. Un juego de este tipo puede definirse formalmente como una clase de **problemas de búsqueda** con los componentes siguientes:

- **El estado inicial**, incluye el estado del juego (la posición de un tablero por ejemplo) y el jugador que mueve.
- **Una función sucesor()**, devuelve una lista de parejas (movimiento, estado), indicando un movimiento legal y el estado que resulta de aplicar el movimiento.
- **Un test terminal**, que determina cuándo se termina el juego. A los estados donde el juego se ha terminado se les llaman estados terminales.
- **Una función utilidad():** también llamada función objetivo, función de rentabilidad o heurística, que da un valor numérico a los estados del juego. Es decir, mide numéricamente como de bueno/malo es un estado del juego para cada jugador.

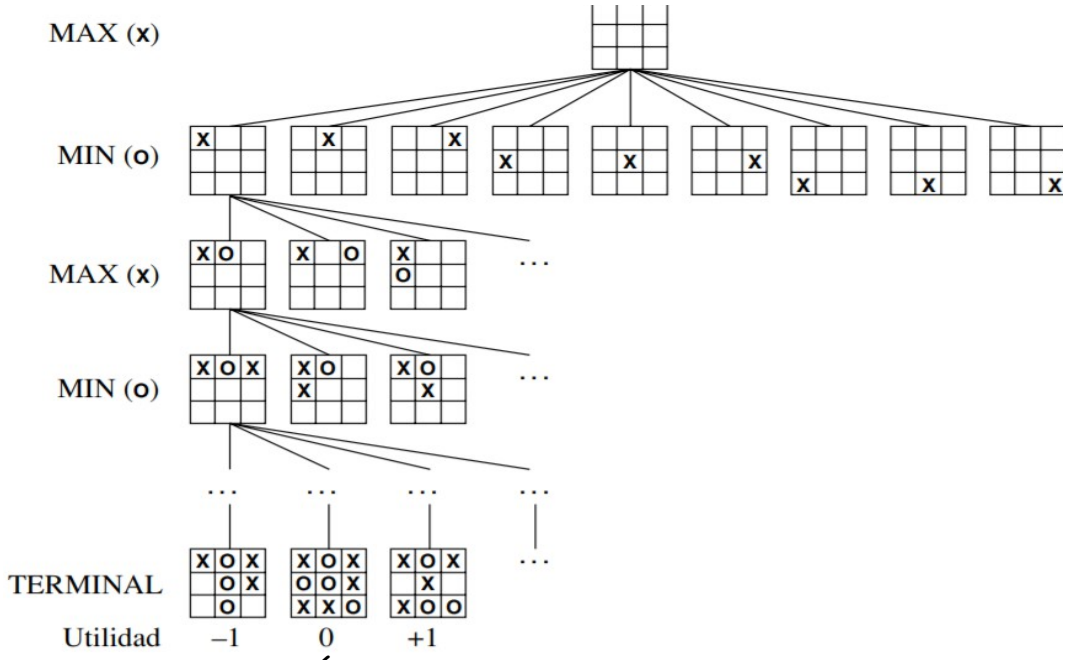


Figura 1: Árbol de juego parcial del juego 3 en raya.

El estado inicial y los movimientos legales para cada jugador definen **un árbol de juego**. La Figura 1 muestra parte de un árbol de juego para el juego de 3 en raya en el que sale MAX. Desde el estado inicial (tablero vacío) que está en la raíz del árbol de juego, MAX tiene 9 movimientos posibles. MAX coloca letras X y MIN coloca letras O en el tablero, hasta que alguno consiga 3 letras en línea (horizontal, vertical o diagonal) o todos los cuadros estén llenos. El número bajo cada nodo hoja indica el valor de la utilidad() de ese estado terminal desde el punto de vista de MAX.

La función **utilidad()** mide lo bueno que es un estado para MAX, debe devolver un valor más alto cuanto más ventajoso sea para MAX ese estado y más bajo cuanto más perjudicial sea para MAX. Si observas, en cada nivel del árbol del juego, es el turno de uno de los jugadores.

El trabajo de MAX (el programa), consiste en escoger la jugada que **maximice la función utilidad()** teniendo en cuenta que cuando sea el turno de MIN, escogerá la jugada que peor le vaya, es decir, MIN si juega de forma óptima, elige la opción que minimiza utilidad(). De ahí el nombre del algoritmo: MIN-MAX.

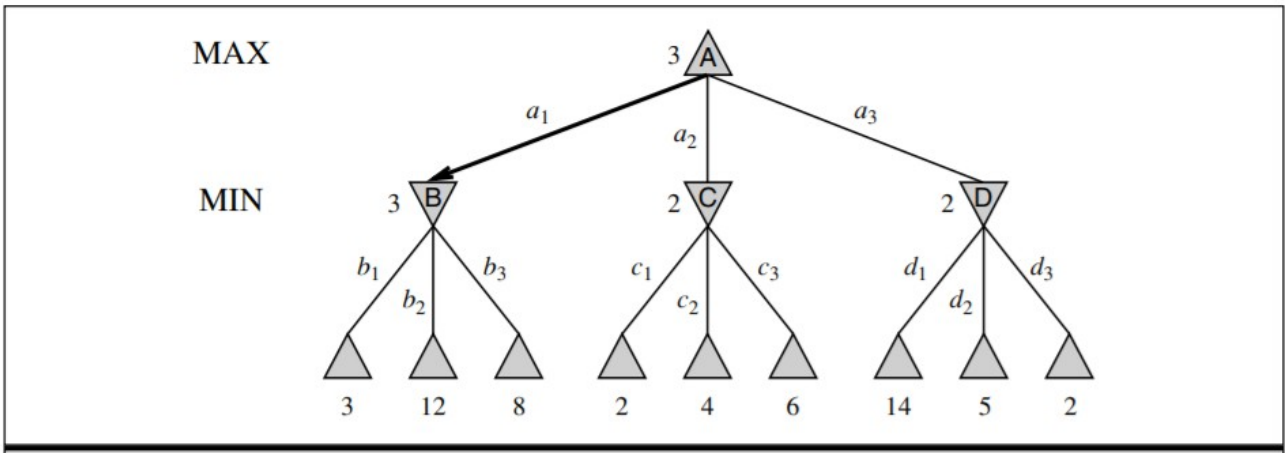


Figura 6.2 Un árbol de juegos de dos capas. Los nodos △ son «nodos MAX», en los que le toca mover a MAX, y los nodos ▽ son «nodos MIN». Los nodos terminales muestran los valores de utilidad para MAX; los otros nodos son etiquetados por sus valores minimax. El mejor movimiento de MAX en la raíz es a_1, porque conduce al sucesor con el valor minimax más alto, y la mejor respuesta de MIN es b_1, porque conduce al sucesor con el valor minimax más bajo.

Algunas aclaraciones sobre este algoritmo que te conviene conocer:

- El algoritmo no es más que un algoritmo de búsqueda en profundidad en el árbol de juego (desde el estado inicial hasta los estados terminales). Lo que busca es una estrategia de juego, **un camino** formado por las jugadas que maximizan la utilidad para MAX y la minimizan para MIN.
- **Realmente el árbol de juego no hay que generarlo como tal**, lo que ocurre es que la lógica del programa lo va recorriendo horizontalmente mediante bucles (aplicar las jugadas posibles de cada nodo conducen a los sucesores de ese nodo) y mediante llamadas recursivas (de forma vertical) explora esos sucesores.
- La función de **utilidad()** puede ser muy sencilla y eso obliga a que el programa deba explorar el árbol a mucha profundidad, o ser muy compleja y permitir al programa

explorar a menos profundidad y mantener su efectividad en el juego.

- Algunos juegos tienen tantas posibles jugadas en cada estado (el ajedrez por ejemplo), que es imposible explorar todos los nodos terminales. Por este motivo, en estos juegos, las llamadas recursivas (la profundidad hasta la que se mira en el árbol) se limitan a cierta cantidad de profundidad, o se limitan por tiempo, o por las dos cosas. A cambio, se perfecciona la función de utilidad() o se añaden trucos para compensar esa falta de información.

EL ALGORITMO MINMAX(n) // n es un estado (un tablero del juego y el turno)
SI n es un estado terminal ENTONCES DEVOLVER utilidad(n); FINSI
SI n es un estado en que mueve MAX ENTONCES
S <- Sucesores(n) //Cada Si de S indica como queda n al hacer la jugada i
DEVOLVER máximo de hacer MINMAX(si) a todos los Si sucesores de S
FINSI
SI n es un estado en que mueve MIN ENTONCES
S <- Sucesores(n) //Cada Si de S es como queda n al hacer la jugada i
DEVOLVER mínimo de hacer MINMAX(si) a los Si sucesores de S
FINSI

A) (0,5 PUNTOS) Te presento un listado de un programa Java que juega a las 3 en raya. Lo copias y lo pruebas. Examínalo e intenta comprender como funciona. Localiza cada una de las partes que se comentan en el texto para responder las siguientes preguntas:

- ¿Cómo se elige el jugador que comienza la partida?
- ¿En qué estados puede encontrarse una partida y qué tipo de dato lo representa?
- ¿Cómo se representa un tablero de una partida en el programa?
- ¿Cómo se representa un estado del juego (tablero y jugador que mueve)?
- ¿Cómo se llama el método que en el programa hace el papel de la función sucesores(n)?
- ¿Cómo crees que es la función utilidad() del código, sencilla o compleja?
- ¿Cómo busca una estrategia de juego? Primero busca en profundidad en el árbol de juego y luego en anchura, o al contrario.

```java
import java.util.Scanner;

public class E51 {

    static Scanner teclado = new Scanner( System.in );
    static int cuentaEjecuciones= 0;

    final static char PIEZAMAX = 'X';
    final static char PIEZAMIN = 'O';
    final static char ESTADOMAX = 'X';
    final static char ESTADOMIN = 'O';

    static char partida= 'J'; //'E'(empate) 'M'(gana MAX) 'm'(gana MIN) 'J'(en Juego)

    public static void main(String[] args) {
        char[][] t = tableroNuevo();
        char turno;
        // Sortear el turno
        System.out.println("\n\nSorteando quien sale primero...");
        double suerte = Math.random();
        System.out.println( tableroToString(t) );
        if(suerte < 0.5) {
```

```
        System.out.println("\nSalgo yo, jeje... Mis piezas " + PIEZAMAX + "\n");
        turno = ESTADOMAX;
    }
    else {
       System.out.println("\nTiene suerte... Usted sale con " + PIEZAMIN + "\n");
       turno = ESTADOMIN;
    }
    String disponibles; // Movimientos disponibles
    while(partida == 'J') {
        disponibles= jugadasDisponibles(t);
        if(disponibles.length() == 0)  // Nadie gana y ya no se puede mover
           partida = 'E';
        else {
            if(turno == ESTADOMAX) {
                Jugada3enRaya j = valorMinMax( t, ESTADOMAX );
                System.out.println("Muevo a la casilla " + j.j + "\n");
                t[j.j / 3][j.j % 3] = PIEZAMAX;
                if( ganaMAX(t) ) partida = 'M';
                else turno = ESTADOMIN;  // El siguiente turno es de MIN
            }
            else {
                int mov;
                do {
                    System.out.print("\nPuede mover a casillas (" + disponibles +
                                     "). Elija una: ");
                    mov = teclado.nextInt();
                } while( disponibles.indexOf( Integer.toString(mov) ) < 0  );
                t[mov / 3][mov % 3] = PIEZAMIN;
                if ( ganaMIN(t) ) partida ='m';
                else turno = ESTADOMAX;
            }
            System.out.println( tableroToString(t) );
        }
    } // while

    if(partida=='E')
        System.out.println("Bien jugado, hemos empatado.");
    else if(partida == 'M')
        System.out.println("Gano yo. Debe mejorar...");
    else
        System.out.println("Enhorabuena, ha ganado!!.");
} // main


static public char[][] tableroNuevo() {
       char[][] t = { {' ', ' ', ' '}, {' ', ' ', ' '}, {' ', ' ', ' '} };
       return t;
}

static public String tableroToString(char[][] t) {
    String s = "\n POSICIONES      ACTUALMENTE\n" +
     " +---+---+---+   +---+---+---+\n" +
     " | 0 | 1 | 2 |   | " + t[0][0] + " | " + t[0][1] + " | " + t[0][2] + " |\n" +
     " +---+---+---+   +---+---+---+\n" +
     " | 3 | 4 | 5 |   | " + t[1][0] + " | " + t[1][1] + " | " + t[1][2] + " |\n" +
     " +---+---+---+   +---+---+---+\n" +
     " | 6 | 7 | 8 |   | " + t[2][0] + " | " + t[2][1] + " | " + t[2][2] + " |\n" +
     " +---+---+---+   +---+---+---+\n";
    return s;
}

// Genera una cadena con las posiciones donde se pueden poner fichas
static public String jugadasDisponibles(char[][] t) {
       String s = "";
       for(int i = 0; i < 3; i++)
              for(int j = 0; j < 3; j++)
```

```
                    if(t[i][j] == ' ') s += (i * 3 + j);
            return s;
    }

    // Si t es terminal OR maxima profundidad alcanzada return utilidad(t)
    // SI estado == MAX -> return jugada con la máxima utilidad
    // SI estado == MIN -> return jugada con la mínima utilidad
    static Jugada3enRaya valorMinMax(char[][] t, char estado) {
        // ¿terminal? gana alguno/no hay movimientos
        // Comprobar a cuantas posiciones puede mover
        String movimientos = jugadasDisponibles(t);
        Jugada3enRaya j = new Jugada3enRaya();
        j.utilidad = utilidad(t);
        if (movimientos.length() == 0 || j.utilidad == 1 || j.utilidad == -1) return j;
        // Realizar cada movimiento posible y comprobar su utilidad
        if(estado == ESTADOMAX) {  // juega MAX
            Jugada3enRaya mejor = new Jugada3enRaya();
            mejor.utilidad = -1;
            char[][] copia;
            for(int m=0; m < movimientos.length(); m++) {
                copia = tableroClona(t);
                int mov = movimientos.charAt(m) - '0';
                copia[mov / 3][mov % 3] = PIEZAMAX;  // Hacer el movimiento en la copia
                Jugada3enRaya temp = valorMinMax(copia, ESTADOMIN);
                if ( temp.utilidad > mejor.utilidad ) {
                    mejor.j = mov;
                    mejor.utilidad = temp.utilidad;
                }
             }
             return mejor;
        }
        else {  // Estoy en estado MIN
            Jugada3enRaya peor = new Jugada3enRaya();
            peor.utilidad = 1000;
            char[][] copia;
            for(int m=0; m < movimientos.length(); m++) {
                copia = tableroClona(t);
                int mov = movimientos.charAt(m) - '0';
                copia[mov / 3][mov % 3] = PIEZAMIN;  // Hacer el movimiento en la copia
                Jugada3enRaya temp = valorMinMax(copia, ESTADOMAX /* , profundidad */);
             if ( temp.utilidad < peor.utilidad ) {
                 peor.j = mov;
                 peor.utilidad = temp.utilidad;
             }
         } // for
         return peor;
     } // else
}

// Hace una copia del tablero
static char[][] tableroClona(char[][] t) {
    char[][] n = new char[3][3];
    for(int i=0;i<3;i++)
        for(int j=0;j<3;j++)
            n[i][j] = t[i][j];
    return n;
}

// Devuelve true si gana MAX o false en otro caso
static boolean ganaMAX(char[][] t) {
    return t[0][0] == PIEZAMAX && t[0][1] == PIEZAMAX && t[0][2] == PIEZAMAX || //Hor
           t[1][0] == PIEZAMAX && t[1][1] == PIEZAMAX && t[1][2] == PIEZAMAX ||
           t[2][0] == PIEZAMAX && t[2][1] == PIEZAMAX && t[2][2] == PIEZAMAX ||
           t[0][0] == PIEZAMAX && t[1][0] == PIEZAMAX && t[2][0] == PIEZAMAX || //Ver
           t[0][1] == PIEZAMAX && t[1][1] == PIEZAMAX && t[2][1] == PIEZAMAX ||
           t[0][2] == PIEZAMAX && t[1][2] == PIEZAMAX && t[2][2] == PIEZAMAX ||
```

```
                t[0][0] == PIEZAMAX && t[1][1] == PIEZAMAX && t[2][2] == PIEZAMAX || //Diag
                t[2][0] == PIEZAMAX && t[1][1] == PIEZAMAX && t[0][2] == PIEZAMAX;
        }

    // Devuelve true si gana MIN o false en otro caso
    static boolean ganaMIN(char[][] t) {
        return t[0][0] == PIEZAMIN && t[0][1] == PIEZAMIN && t[0][2] == PIEZAMIN || //Hor
                t[1][0] == PIEZAMIN && t[1][1] == PIEZAMIN && t[1][2] == PIEZAMIN ||
                t[2][0] == PIEZAMIN && t[2][1] == PIEZAMIN && t[2][2] == PIEZAMIN ||
                t[0][0] == PIEZAMIN && t[1][0] == PIEZAMIN && t[2][0] == PIEZAMIN || //Ver
                t[0][1] == PIEZAMIN && t[1][1] == PIEZAMIN && t[2][1] == PIEZAMIN ||
                t[0][2] == PIEZAMIN && t[1][2] == PIEZAMIN && t[2][2] == PIEZAMIN ||
                t[0][0] == PIEZAMIN && t[1][1] == PIEZAMIN && t[2][2] == PIEZAMIN || //Dia
                t[2][0] == PIEZAMIN && t[1][1] == PIEZAMIN && t[0][2] == PIEZAMIN;
    }

    // Puntúa tablero y mira lo interesante que es para MAX:-1 pierde,1 gana,0 empate
    static double utilidad( char[][] t ) {
        if( ganaMAX(t) ) return 1;
        if( ganaMIN(t) ) return -1;
        return 0;
    }

}  // clase

class Jugada3enRaya {
        int j;       // Jugada: donde colocar la pieza fila = j / 3 y columna j % 3
        double utilidad;  // Utilidad para MAX
}
```

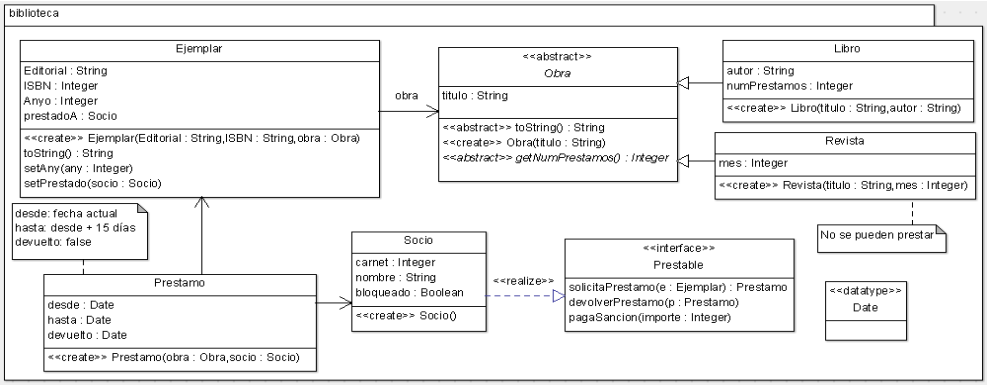
B) (0.5 PUNTOS). Modifica el código, crea una variable estática entera llamada **cuentaEjecuciones** inicializada a 0 y la incrementas en 1 cada vez que se ejecute la función **utilidad()**. Al acabar una partida imprime su valor después de que el programa indique el resultado. Juega 3 partidas, me pasas 3 capturas del resultado final y me indicas de media cuantas veces se ejecuta la función **utilidad()**.

EJERCICIOS JEDI (EL QUE TE TOQUE)

Como siempre, aunque solo tenéis que entregar el que os toca, deberíais intentar hacerlos todos para practicar.

EJERCICIO 6 (1 PUNTO). CLASES ABSTRACTAS.

Abre el fichero .pdf que contiene un diagrama de clases usando UML 1.4 realizado con el programa ArgoUML que también puedes descargar. Intenta implementar en Java lo que expresa el diagrama UML del que tienes una imagen a continuación.



Crea una clase ejecutable que cree dos libros, una revista, un ejemplar de cada libro y de la revista, un socio, que el socio haga 2 préstamos y devuelva uno.

EJERCICIO 7. IDENTIFICAR E IMPLEMENTAR CLASES.

Quieres realizar una aplicación para realizar un análisis de riesgos cuantitativo de un sistema informático aplicando **MAGERIT** (**Metodología de Análisis y Gestión de Riesgos TIC**).

> **Nota:** El objetivo de este ejercicio es identificar e implementar clases de un problema y además usar una técnica de resolución de problemas aplicando unos algoritmos especiales que realizan una búsqueda exhaustiva de la solución a un problema, llamados **algoritmos greedy** (adjetivo inglés que significa avaricioso o voraz), de los que te indico en pseudocódigo de alto nivel su funcionamiento:

```
// Entradas: C es un conjunto de soluciones candidatas
// Salida: S un conjunto de soluciones al problema
ALGORITMO Greedy( C ): devuelve S
   S = { }         // Partimos de un conjunto de soluciones inicial (puede estar vacío)
   MIENTRAS haya candidatos en C HAZ
       Seleccionar un candidato x de C
       C <- C – {x}          // Quitar x a C
       SI  S unión con {x} es una solución posible
       ENTONCES
            S <- S U {x}
       FINSI
   FINMIENTRAS
   DEVOLVER S
FINALGORITMO
```

PASO 1 (0.5 PUNTOS). LEE EL TEXTO Y COMPLETA LOS EJERCICIOS.

En este modelo de análisis de riesgos, se trabaja con valores que son números reales, siempre superiores o iguales a cero. Un análisis de riesgos consiste en:

(1) INVENTARIAR ELEMENTOS.

- **Inventariar activos**: un activo es cualquier elemento del sistema que tenga utilidad para la organización. Cada activo tiene: un código (puede ser la posición que ocupe en un array), un nombre, un valor (o económico o importancia en una escala de 0-10 ó de 0-100) y uno o varios tipos. Hay flexibilidad a la hora del detalle con el que definir activos: un activo podría llamarse "Equipo" y representar el hardware y el sistema operativo y todas las aplicaciones de cualquier PC. También podrías tener un activo llamado "teclado" de tipo hardware que represente un teclado de un equipo, otro para el disco duro, la placa, etc.
  Los activos pueden ser de uno o varios de estos tipos: **'H'** (hardware), **'S'** (software), **'D'** (datos), **'E'** (servicios), **'C'** (comunicaciones), **'M'** (medios de almacenamiento: un CD...), **'A'** (sumunistros auxiliares como electricidad, papel, tinta), **'P'** (personal) y **'L'** (lugares o instalaciones, como un edificio).
- **Inventariar las amenazas**: y otras muchas cosas además de las amenazas, que no vamos a usar para simplificar el ejercicio.

> **Nota**: Puedes ver el catálogo real de la metodología en este enlace: https://www.ccn-cert.cni.es/documentos-publicos/1791-magerit-libro-ii-catalogo/file.html

(2) DEFINIR DEPENDENCIAS DIRECTAS ENTRE ACTIVOS

Después de inventariar activos, amenazas, salvaguardas, niveles de implantación, definir perfiles y estrategisas de gestión, etc. que tiene el sistema, interesa saber si un activo A depende o no de otro activo B y en qué grado depende. El grado de dependencia es un número entre 0.0 (no hay dependencia) y 1.0 (dependencia absoluta). Hay dos tipos de dependencias: directas (las debe indicar el usuario) e indirectas (las debe calcular el programa).

- A depende de B de forma directa se escribe: **A -> B** y tiene un grado asociado (0-1)
- A depende de C de forma indirecta se escribe: **A => C**. Y significa que debe existir al menos otro activo B intermedio que cumpla (A => B) y (B -> C), es decir debe haber otro del que A depende directa o indirectamente y ese otro depende de C.

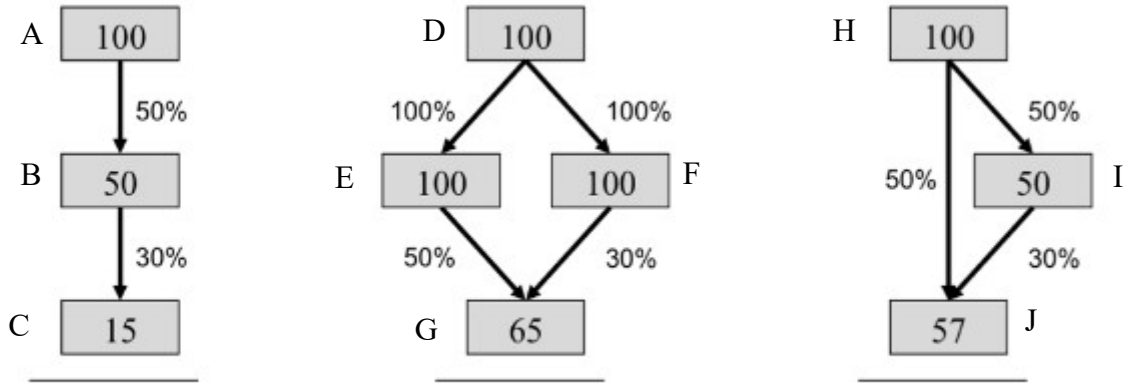
El grado de dependencia de dos activos que dependan de forma indirecta se calcula como:

grado(A => C) = suma en i(grado(A => Bi) * grado(Bi -> C))

Donde i es el número de caminos entre A y C, y el grado de cada camino es el producto de los pesos de las dependencias. Las sumas de los grados de cada camino se realizan de acuerdo a esta fórmula:

sumaDePareto(a, b) = 1 – (1 - a) * (1 - b)

Ejemplos: Para que comprendas la forma de calcular este grado de dependencia.



PRIMER CASO (IZQUIERDA)

Valores de activos: A = 100 B = 50 C = 15

Dependencias directas: A -> B con grado 50% = 50/100 = 0.5

B -> C con grado 30% = 30/100 = 0.3

A=>C porque A -> B -> C ¿grado de A => C?

¿Por cuántos caminos se puede llegar saliendo de A hasta C? Uno. Pues eso es la i de la fórmula (el nº de caminos) y el camino es: **A -> B -> C**

El grado es el producto de grados del camino: A->B (0,5) y B->C (0,3) = 0,5 * 0,3 = 0,15

Como no hay más caminos, 0,15 es el grado de A => C.

SEGUNDO CASO (CENTRO)

Valor de activos: D = 100 E = 100 F = 100 G= 65

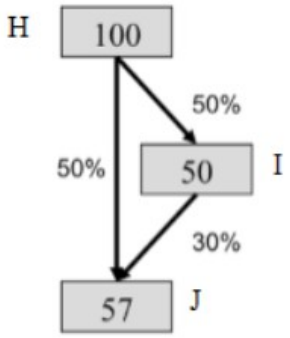
Dependencias directas: D -> E con grado 1 y E -> G con grado 0.5

D -> F con grado 1 y F -> G con grado 0.3

¿grado de D => G?
Hay 2 caminos que salen de D y llegan hasta G (i=2):
Primer camino: D -> E -> G = 1.0 * 0.5 = 0.5
Segundo camino: D -> F -> G = 1.0 * 0.3 = 0.3
La dependencia será la suma de pareto entre ellos:
pareto(0.5 , 0.3) = 1 - (1 – 0,5) * (1 – 0,3) = 1 – 0,5 * 0,7 = 1 – 0,35 = 0,65 = 65%

(0.1 PUNTOS) HAZ EL CASO DE LA DERECHA Y ENTREGA EL RESULTADO

Grado de dependencia indirecta entre los activos H y J: ¿ H => J ?
¿Cuántos caminos hay desde H hasta J? (e indica cuales son)
Para cada camino, calcular el producto de los pesos:
Por último hacer la suma de pareto entre ellos:



Lo que debe crear el programa, es un **grafo dirigido** (especie de red) donde cada nodo es un activo y cada flecha indica que el activo del que sale la flecha depende del activo al que apunta la flecha en cierto %. Al hacer estos grafos se cae en errores si se crean ciclos (un activo no debería depender de forma indirecta de sí mismo). El usuario debe poder definir grafos de este tipo y la pregunta es: ¿Cómo representar en un programa estos grafos? Para el grafo de la figura de la derecha:

Array de activos:

H	I	J

	H	I	J
H		0,5	0,5
I	0		0,3
J	0	0	

Tabla 5.1: Grafo de dependencias directas de la figura representadas mediante array 2D.

Bueno, la forma más sencilla es tener numerados los n activos del análisis con números enteros de 0 a n-1 y usar un array 2D de doubles llamado **dD /* dependencias directas */** con tantas filas y columnas como activos haya. En principio todas las celdas están a 0.0 (no depende nada de nada) y cuando se define una dependencia, se indica el Id del activo dependiente (A) y el Id del activo del que depende directamente (B) y se anota en la celda dD[a][b] el grado.

Si observas la tabla, la diagonal principal (cuando fila es igual a columna) están sombreadas para indicar que no hay que definir dependencias (el programa debe impedir que se indiquen, no se van a usar nunca). Las letras lógicamente, no se almacenan, las pongo en la imagen para que se vea a qué activo corresponde, serían el nº de fila y columna.

Ahora, el análisis de riesgos debe definir otra tabla 2D llamada por ejemplo **dI** (**dependencias indirectas**) y calcular las indirectas a partir de las directas y al mismo tiempo, comprobar que no crea ningún ciclo, en cuyo caso los datos indicados por el usuario serían incorrectos. Y la pregunta vuelve a ser ¿Cómo hacerlo?

De la misma forma que has hecho en los ejercicios anteriores, primero debes averiguar para cada activo A, por cuantos diferentes caminos puedes ir (sin bucles) hasta otro activo

siguiendo las flechas de las dependencias. Aquí es donde entra el algoritmo greedy.

Lo primero que debes hacer es pensar como vas a representar cada camino. Yo te aconsejo que uses strings de la forma "nºA -> nºB" (el motivo es que crear nuevos caminos y encontrar bucles en uno es muy sencillo). Por ejemplo comprobar si se va a generar un bucle en un camino antes de añadir el activo nº3 (un nuevo paso del camino):

```
SI camino.indexOf( "3" ) >= 0
ENTONCES hayBucle<- true
SINO nuevoCamino= camino + " -> 3"
```

El algoritmo para encontrar todos los caminos saliendo de un Activo A:

```
// Entradas: un activo A y una tabla de dependencias directas dD.
// Salidas: S conjunto de caminos que pueden recorrerse desde A
ALGORITMO GeneraCaminos(A, dD)
// S inicialmente contiene tantos caminos como dependencias directas salgan de A
PASO 0:PARA x=0 HASTA dD[A].longitud-1   HAZ
           SI dD[A][x] > 0 ENTONCES S = S +  { "A -> x" } FINSI
       hayBucles <- false
       continuar <- true
       Bucle <- ""
// Mientras haya cambios en el resultado S y no haya bucles, seguir
       hastaCam <- -1  // Último camino que hay pendiente de explorar
PASO 1:MIENTRAS continuar AND NOT hayBucles HAZ
PASO 2:    continuar <- false;
           desdeCam <- hastaCam + 1;
           hastaCam <- S.length - 1;
PASO 3:    PARA c = desdeCam HASTA hastaCam (c es de la forma "A=>B") HAZ
PASO 4:        B <- Activo en que acaba el camino c
PASO 5:        PARA d = cada dependencia directa de B en dD de la forma B -> Y
PASO 6:            SI dD[B][Y] > 0 ENTONCES
PASO 7:                SI c.indexOf(Y) >= 0 ENTONCES
PASO 8:                    hayBucles <- true
PASO 9:                    bucle <- c + " -> Y"
PASO 10:                   break;
                       SINO
PASO 11:                   S <- S + { c + " -> Y" }
PASO 12:                   continuar <- true
                       FINSI
                   FINSI
               FINPARA
PASO 13:       SI hayBucles ENTONCES break; FINSI
           FINPARA
       FINMIENTRAS
       DEVUELVE S
FIN ALGORITMO
```

Una vez que tengas todas las dependencias de cada activo A, debes procesar los caminos para rellenar las dependencias indirectas en la tabla **dI[A]** (que inicialmente deben estar a 0). Debes saltar **dI[A][A]** porque no tienen sentido.

Debes recorrer cada camino de S y calcular el grado de cada dependencia (los tienes en la tabla dD[][]). Recorres cada camino de S (estarán los directos e indirectos) y almacenar en dI[A][ultimo] siendo ultimo el último activo del camino el resultado. El resultado será el producto de cada grado "X -> Y" que está en dD[X][Y]. Algoritmo:

```
PASO 1 PARA c= cada camino de S HAZ
PASO 2        grado = 1.0;
PASO 3        PARA cada "X -> Y" de c HAZ
PASO 4              grado <- grado * dD[X][Y]
```

```
PASO 5          FINPARA
PASO 6          ultimo <- último activo del camino c
PASO 7          SI dI[A][ultimo] = 0.0
PASO 8          ENTONCES dI[A][ultimo] <- grado
PASO 9          SINO dI[A][ultimo] <- suma_pareto( dI[A][ultimo], grado)
PASO 10         FINSI
PASO 11FINPARA
```

Esto hay que repetirlo para cada cada activo A del análisis de riesgos. Cada vez que trabajes con uno, su fila de la tabla dI debe quedar calculada.

IMPLEMENTA EN CÓDIGO.

(0.6 PUNTOS) Debes entregar los ficheros que definan las clases: Activo y AnalisisRiesgos de forma que un análisis de riesgos pueda:

- Manejar hasta 10 Activos en el array activos.
- Definir una tabla de dependencias directas llamada dD de 10x10.
- Definir una tabla de dependencias indirectas dI de 10x10.
- Pedir al usuario por consola las dependencias directas entre los activos con el método leerDependencias()
- Calcular las dependencias indirectas a partir de las directas con el método calculaDI() .

(0.2 PUNTOS) También debes crear un programa que use esas clases para ofrecer esta funcionalidad:

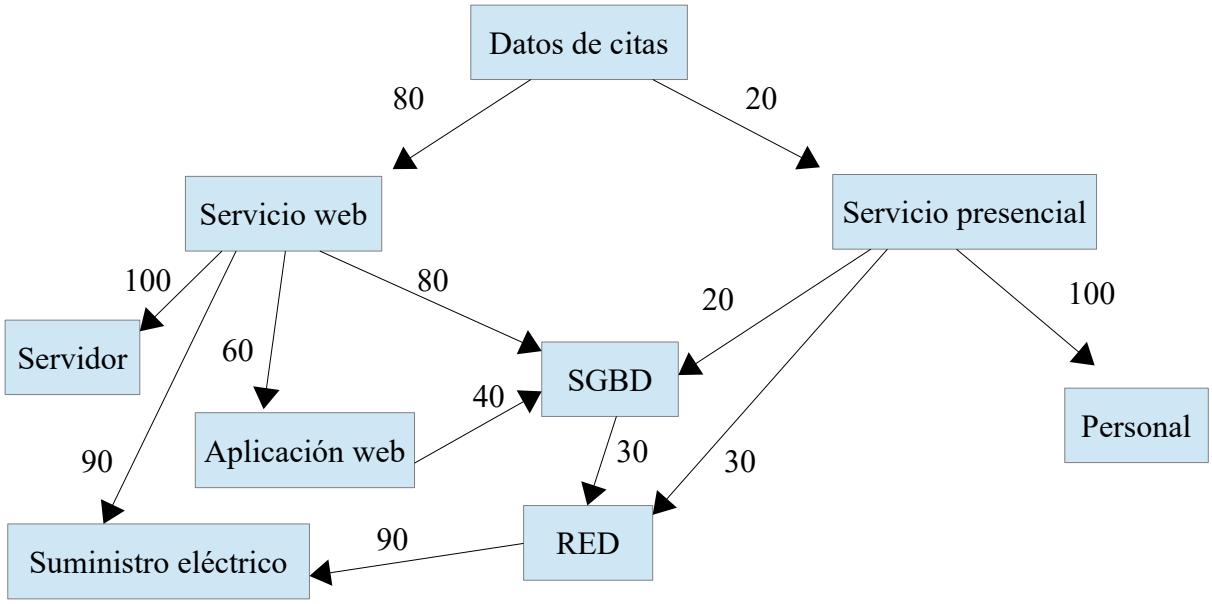
(1) Crear un análisis de riesgos

(2) Ofrecer un menú para:

1. Definir Activos del sistema.
2. Definir dependencias directas entre activos.
3. Calcular y mostrar dependencias directas e indirectas.
4. Salir.

(0.1 PUNTOS) Entrega una captura de pantalla de mostrar la tabla de dependencias indirectas de este caso:

Una empresa dedicada a realizar ITV, analiza el riesgo de su sistema de recogida de citas.



Activos:

- Datos de las citas con los clientes. Valor 100. Tipo 'D'
- Servicio Web: valor 90 tipo 'E'
- Servicio presencial: valor 10 y tipo 'E'
- Servidor: valor 30 y tipo hardware, software y media.
- Aplicación web: valor 40 y tipo software.
- SGBD: tipo software valor 90.
- Red: tipo comunicaciones, hardware y software y valor 30.
- Sumninistro eléctrico: suministro, valor 20 y tipo 'A'.
- Personal: tipo personal y valor 30.

Nota: *como hay muchos activos y dependencias, si los defines como constantes en los arrays para probar, no tendrás que teclearlas cada vez.*

EJERCICIO 8. CLASES ABSTRACTAS E INTERFACES.

Crea un clase CuentaBancaria que tenga el saldo y un nombre de titular y un array (digamos de tamaño 10) llamado movimientos y que almacene objetos de tipo OperacionBancaria. Crea el contructor y sobreescribe toSting() y proporciona setter para el saldo.

Ahora crea una clase abstracta llamada **OperacionBancaria** que tenga los datos importeAnterior, importePosterior, fecha y hora (la fecha en que se aplica), la cuenta bancaria en que se aplica y la operación abstracta aplicar(CuentaBancaria). El constructor por defecto es OperacionBancaria(cuenta, importe) la implementación de aplicarla depende de cada tipo de cuenta.

Crea varias clases hijas de OperacionBancaria e implementa en todas ellas toString() que debe imprimir algo así como: ["TIPO", FECHA, IMPORTE, SALDOANTERIOR, SALDOPOSTERIOR] y aplica() que debe añadirla en el array de movimientos de la cuenta

- **Depósito**: se añade dinero a una cuenta.
- **Transferencia**: se mueve dinero desde una cuenta origen a una cuenta destino. Tiene además la cuenta de destino (se conoce en el momento de aplicarla) del dinero y un booleano que indica si está anulada.
- **Retirada**: se extrae dinero de la cuenta bancaria. Si el importe que se saca es mayor que el saldo de la cuenta, se crea un Prestamo a devolver en 1 año y .
- **Prestamo**: tiene además un nº de préstamo (se genera automáticamente según un contador que tiene la clase), el importe pendiente de amortizar y el plazo de devolución, y la operación amortizar(importe).
- **Amortizar:** contiene el nº de préstamo que se amortiza.

Crea la interfaz Anulable que contenga una única operación llamada anular(). Haz que la clase Transferencia implemente la interfaz anulable y la implementas. No olvides actualizar los saldos y añadir el movimiento de anulación de las cuentas que participan.

Ahora crea la clase Cliente y un método main para poder ejecutarla. Crea dos cuentas bancarias, crea y aplica 3 depósitos del importe que quieras, crea y aplica una transferencia de una a otra, haz una retirada superior al saldo que tengas, e imprime la cuenta (que debe imprimir también las operaciones que se han producido).

EJERCICIO 9 (1 PUNTO). CLASES INTERNAS.

Las clases internas tienen sentido cuando una clase solamente la va a utilizar otra. En ese caso, no tiene sentido dejar a esa clase fuera de donde se va a utilizar. Así que se define dentro de la otra, por eso es interna. Implementa estos escenarios en los proyectos U0509A, U0509B, ... y me pasas el código de cada una.

a) **CLASE INTERNA.** La clase **Motor** solamante la va a utilizar la clase **Vehiculo**. La clase Motor tiene:

- potenciaCV: un double indicando los caballos de vapor de potencia.
- Un constructor al que se le pasa la potencia.
- **double velocidadKmh(segundos)**: devuelve la velocidad en Kmh transcurridos los segundos pasados aplicando esta fórmula, donde el peso es de la clase Vehículo:

  Kmh= 1.60934 (0.68 * (raiz_cuadrada(32.17 * 2 *potencia *segundos 32.17/peso)))

La clase Vehiculo tiene:

- El motor que usa.
- Un peso.
- El contructor al que se le pasa peso y potencia.
- El método getVelocidad(segundos)

b) CLASE INNER LOCAL: Si te fijas en la clase Motor, realmente solamente se utiliza dentro del método getVelocidad() de Vehiculo. Si el diseñador cree que eso siempre va a ser así, puede pensar que no tiene sentido que la clase exista fuera del método. Así que la define dentro del método. Copia y modifica el código del ejercicio anterior para adaptarlo a esta situación.

c) CLASES INNER ANÓNIMAS: Imagina que en cierto momento no nos interesa calcular la velocidad con el método que tiene definida la clase interna y para algunos vehículos, queremos usar su propio método. Imagina que en ese momento queremos calcularlo como tiempo/potencia. Lo que vas a hacer es sobreescribir la implementación actual del método en la llamada siguiendo este esquema cuando contruyes el objeto vehiculo desde el programa principal:

```
:
Vehiculo v1 = new Vehiculo(...) { <sobreescribir el método aquí>  }
:
```

d) CLASES INTERNAS ESTÁTICAS. Una clase interna estática es aquella interna que solamente puede acceder a elementos estáticos de la clase que la contiene. Ten en cuenta eso y convierte la clase Motor en una clase interna static. Tendrás que hacer algún cambio porque ya no podrá acceder a peso del vehículo donde está.

e) INTERFACES FUNCIONALES. Dada la siguiente interfaz,

```
interface Motorizado {
        double getVelocidad(tiempo, peso, potencia);
        String toString();
}
```

Prueba la anotación **@FuncionalInterface** para asegurarte de que es una interfaz funcional Y en caso de que lo sea, crea una nueva versión del método main() donde crees variables de esa interfaz usando una función lambda en vez de una clase inner anónima.

EJERCICIO 10. DEFINIR CLASE ABSTRACTA O INTERFAZ.

Define una clase abstracta o una interfaz de un juego del tipo que hemos visto en el ejercicio 5 (por turnos, de dos jugadores y con información completa) que implementará el algoritmo MINMAX para que el programa elija jugada).

A partir de ella crea un juego Conecta4 que consiste en un tablero de 6x7 casillas donde cada jugador va dejando sus piezas con el objetivo de conseguir dejar 4 fichas consecutivas en horizontal, vertical o cualquiera de las diagonales (pero no implementes detectar las diagonales si no quieres). Haz una versión MIN-MAX del juego conecta 4.

EJERCICIOS OPCIONALES

Como siempre, aunque solo tenéis que entregar el que os toca, deberíais intentar hacerlos t

EJERCICIO OPCIONAL (+4 PUNTOS).

Haz una versión mejorada del código del ejercicio 5 usando PODA alfabeta o límite de profundidad + mejora de la función utilidad().

(+2 PUNTOS) MEJORA utilidad() Y LIMITA PROFUNDIDAD A 4.

Ahora la búsqueda no devuelve 0 en caso de que no gane ninguno. Devuelve un valor más elaborado que mide las posibiliades de ganar de MAX. Esto permite no tener que llegar a explorar hasta el final de las partidas, con explorar 4 niveles ya nos haremos una idea de como van a ir las cosas (al menos en teoría). Cambiar el **return 0** de la función **utilidad()** por este peudocódigo:

```
// Calcular para MAX cuantas líneas puede hacer aún:
Hacer una copia del tablero.
Rellenar los espacios vacíos del tablero con sus piezas.
sumaMax <- Contar cuantas líneas de 3 salen + 1.0 // Le sumas 1 para que nunca sea 0
// Calcular para MIN cuantas líneas puede hacer aún:
Hacer otra copia del tablero.
Rellenar los espacios vacíos del tablero con las piezas de MIN.
sumaMin <- Contar cuantas líneas de 3 salen + 1.0
DEVOLVER sumaMax / sumaMin;  // Relación de lo bueno que es para MAX
```

Para limitar la profundidad de la exploración vertical, añadimos un parámetro entero a la función recursiva **valorMinMax()** llamado **profundidad**. Nada más entrar, la función le resta 1. Si llega a 0 lo considera como un estado terminal y le calcula la utilidad(). La primera vez que se llama a la función la llamada será:

valorMinMax(t, ESTADOMAX, 4);

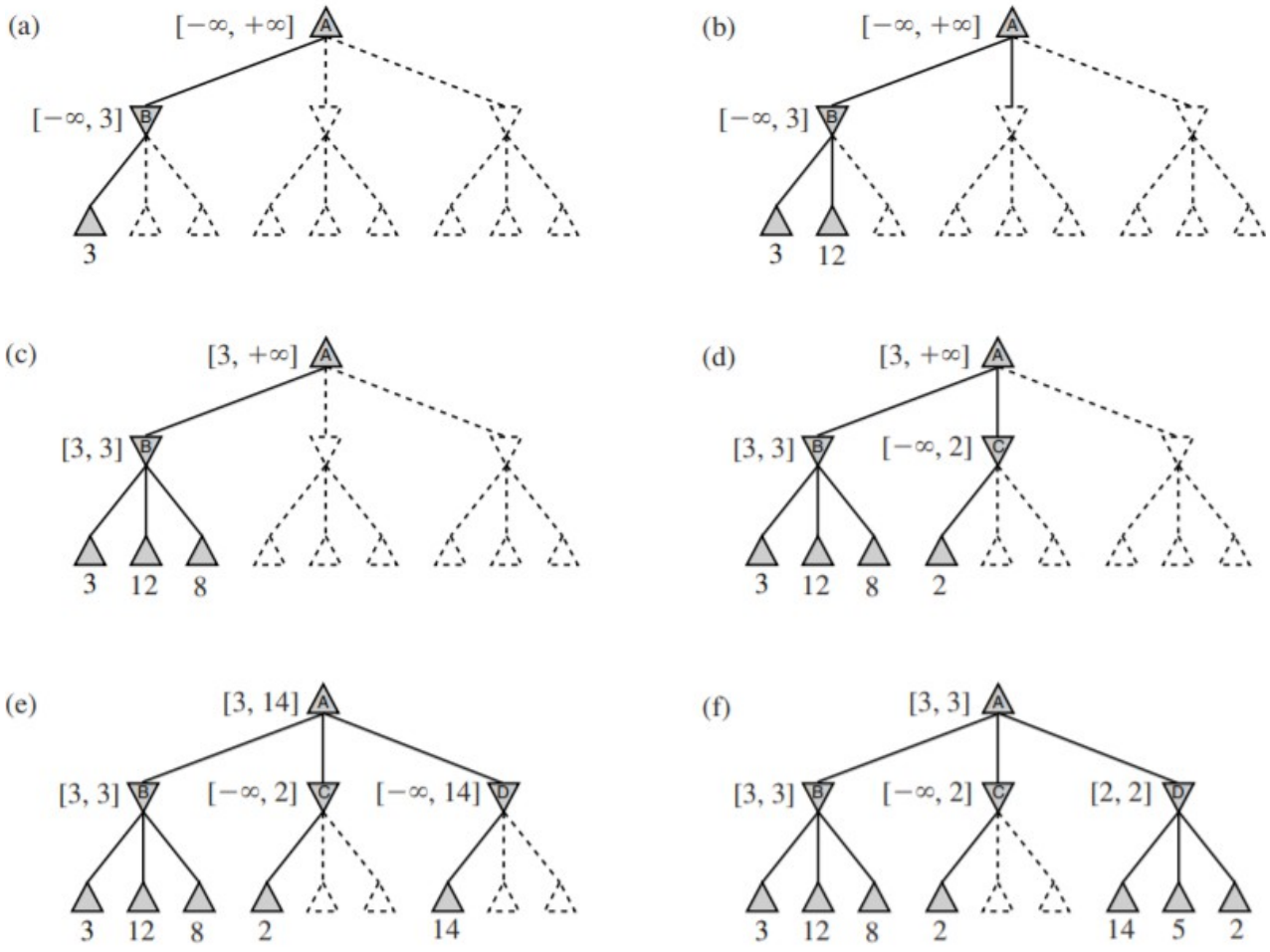
(0.2 PUNTOS) Cuando lo tengas modificado juega unas partidas y responde:

- ¿Es igual de competente el programa que antes? Indica de las partidas que has jugado ¿Qué % ha perdido el programa de cuantas?
- ¿Cuántas veces se ejecuta la función utilidad() ? ¿Más o menos que antes?

- ¿Es más rápido en responder o más lento?

(+ 2 PUNTOS) ALGORITMO PODA ALFA-BETA (αβ-PRUNING).

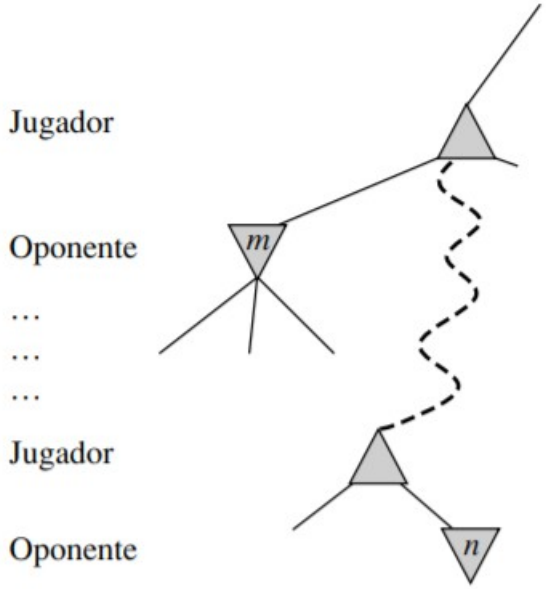
El problema de la búsqueda minimax es que el número de estados que tiene que examinar crece de forma exponencial con en el número de movimientos de cada estado. Lamentablemente no podemos eliminar ese exponente sin cambiar el juego, pero si podemos dividirlo, con eficacia, en la mitad. La jugada o el truco, consiste en que es posible calcular la decisión minimax correcta sin mirar todos los nodos en el árbol de juegos. Es decir, podemos tomar prestada la idea de podar el árbol de posibilidades:



En la Figura se visualiza las etapas en el cálculo de la decisión óptima para el árbol de juegos de la Figura 2. En cada punto, mostramos el rango de valores posibles para cada nodo. (a) La primera hoja debajo de B tiene el valor 3. De ahí B, que es un nodo MIN, tiene un valor de como máximo 3. (b) La segunda hoja debajo de B tiene un valor de 12; MIN evitaría este movimiento, entonces el valor de B es todavía como máximo 3. (c) La tercera hoja debajo de B tiene un valor de 8; hemos visto a todos los sucesores de B, entonces el valor de B es exactamente 3. Ahora, podemos deducir que el valor de la raíz es al menos 3, porque MAX tiene una opción digna de 3 en la raíz. (d) La primera hoja debajo de C tiene el valor 2. De ahí, C, que es un nodo MIN, tiene un valor de como máximo 2. Pero sabemos que B vale 3, entonces

MAX nunca elegiría C. Por lo tanto, no hay ninguna razón para continuar explorando a los otros sucesores de C. Este es un ejemplo de la poda de alfa-beta. (e) La primera hoja debajo de D tiene el valor 14, entonces D vale como máximo 14. Este es todavía más alto que la mejor alternativa de MAX (es decir, 3), entonces tenemos que seguir explorando a los sucesores de D. Observa también que ahora tenemos límites en todos los sucesores de la raíz, entonces el valor de la raíz es también como máximo 14. (f) El segundo sucesor de D vale 5, así que otra vez tenemos que seguir explorando. El tercer sucesor vale 2, así que ahora D vale exactamente 2. La decisión de MAX en la raíz es moverse a B, dando un valor de 3.

A esta técnica se le llama poda-alfabeta. Cuando lo aplicamos a un árbol minimax estándar, devuelve el mismo movimiento que devolvería minimax, ya que podar las ramas no puede influir en la decisión final.
La poda alfa-beta puede aplicarse a árboles de cualquier profundidad, y, a menudo, es posible podar subárboles enteros. El principio general es: considera un nodo n en el árbol, tal que el Jugador tiene una opción de movimiento a ese nodo. Si el Jugador tiene una mejor selección m en el nodo padre de n o en cualquier punto más lejano, entonces n nunca será alcanzado en el juego actual. Una vez que hemos averiguado bastante sobre n (examinando a algunos de sus descendientes) hasta alcanzar esta conclusión, podemos podarlo. Recuerda que la búsqueda minimax es primero en profundidad, así que en cualquier momento solamente tenemos que considerar los nodos a lo largo de un camino en el árbol.



La poda alfa-beta consigue su nombre de los dos parámetros que describen los límites sobre los valores hacia atrás que aparecen a lo largo del camino: **alfa es el valor de la mejor opción del camino (es decir, valor más alto) que hemos encontrado hasta ahora en cualquier punto elegido a lo largo del camino para MAX**. Y **beta** es el valor de la mejor opción (es decir, valor más bajo) que hemos encontrado hasta ahora en cualquier punto elegido a lo largo del camino para MIN. La búsqueda alfa-beta actualiza el valor de alfa y beta y según se va recorriendo el árbol y poda las ramas restantes en un nodo (es decir, termina la llamada recurrente) tan pronto como el valor del nodo actual es peor que el actual valor para MAX o MIN, respectivamente.

```
// al principio alfa es -infinito y beta es +infinito
```

```
EL ALGORITMO MINMAX(n, jugador, alfa, beta)
    SI n es un estado terminal ENTONCES  DEVOLVER utilidad(n); FINSI
    SI n es un estado en que mueve MAX ENTONCES
        PARA cada ni de Sucesores(n)   HACER
            v <- máximo( v, MINMAX(ni, MIN, alfa, beta) )
            SI v >= beta ENTONCES DEVOLVER v; FINSI;
            alfa <- máximo( alfa, v)
        FINPARA
        DEVOLVER v
    FINSI
    SI n es un estado en que mueve MIN     ENTONCES
        PARA cada ni de Sucesores(n)   HACER
            v <- mínimo( v, MINMAX(ni, MAX, alfa, beta) )
            SI v <= alfa ENTONCES DEVOLVER v; FINSI;
            beta <- mínimo( beta, v)
        FINPARA;
        DEVOLVER v
    FINSI
```

Modifica el programa original para que utilice la poda alfa beta. Cuando lo tengas modificado me pasas el código (0.7 PUNTOS) juega unas partidas y me pasas capturas de pantalla del final (0.1 PUNTOS) y respondesi a estas preguntas (0.2 PUNTOS):

- Es igual de competente el programa? Indica de las partidas que has jugado, su número y qué % ha perdido?
- ¿Cuántas veces se ejecuta la función utilidad() ? ¿Más o menos que antes?
- ¿Es más rápido en responder o más lento?